

1.Unsold Products

In a coding project you are developing a feature for a data visualization tool that processes large datasets of products stored in array **A**. One dataset contains sales figures for various products, where a sale value of 0 indicates that the product was not sold during a specific period. Your task is to find and return an integer array representing the dataset representing all the unsold products at the end.

Input specifications:

Input1 : An integer array A containing sales number of a particular product.

Output Specification:

Return an integer array representing the dataset representing all the unsold products at the end.

Example1 :

Input1 : {5,2,0,8,0,2,1}

Output : {5,2,8,2,1,0,0}

Explanation :

Here, A={5,2,0,8,0,2,1}. The dataset contains zeros at position 2 and 4. After shifting all zeros to the end, the non-zero elements (5,2,8,2,1) retain their original order. Therefore, {5,2,8,2,1,0,0} will be returned as output.

Move Zeros problem

<https://leetcode.com/problems/move-zeroes/submissions/1533396854/>

2.Count of specific string

Write an algorithm to find the count of the specific character in the given data.

Input

The first line of the input consists of a string data representing the data to be encoded.

The second line consists of a character-coder representing the character to be counted in the data.

Output

Print an integer representing the count of the specific character. If the required character is not found then print 0.

Note

The specific character and given string consists of English letters and special characters only.

Example.

Input:

haveagooodday

Output :

3

Explanation:

The character "a" occurs three times in the data. So the output is 3

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
int main() {
```

```
    string text = "haveagoodday";
```

```
    char targetChar = 'a';
```

```
    int occurrenceCount = 0;
```

```
    // write your code here
```

```
    // Output the result
```

```
    cout << "The character '" << targetChar << "' occurs " << occurrenceCount << " times  
in the text." << endl;
```

```
    return 0;
```

```
}
```

<https://www.geeksforgeeks.org/problems/count-the-characters1821/1>

3. Rearrangement of bits

Alex give you a positive number **N** and wants you rearrange the bits of the number in its binary representation such that all the sets bits are in consecutive order. Your task is to find and return an integer value representing the minimum possible number that can be formed after re-arranging the bits of the number N.

Note: A set bit means 1 in the binary form of a number.

Input Specification :

Input1 : An integer value N.

Output Specification :

Output : An integer value representing the lowest possible number amongst all possible combinations.

Example 1 :

Input1 : 10

Output : 3

Explanation :

Here, the given number is 10, whose binary representation is 1010. So we can rearrange the bits to make all these bits consecutive as 0011, which is the lowest possible number amongst all possible combinations. Therefore, **3** is returned as the output.

Example 2 :

Input1 : 2

Output : 1

```
#include <iostream>

using namespace std;

int rearrangeBits(int N) {

    int count = __builtin_popcount(N); // Count the number of set bits (1s)

    // Create the smallest number using the counted set bits

    int result = (1 << count) - 1; // This forms a number with `count` ones in binary

    return result;

}

int main() {

    int N;

    cin >> N; // Input the number

    cout << rearrangeBits(N) << endl; // Output the result

    return 0;

}
```

4) Even Odd

Jack has an array A of length N. He wants to label whether the number in the array is even or odd. Your task is to help him find and return a string with labels even or odd in sequence according to which the numbers appear in the array.

Input Specification :

Input1 : An integer array A, representing the array of numbers

Input2 : A integer N, representing the length of array.

Output Specification:

Return a string with labels even or odd in sequence according to which the numbers appear in the array according to which the numbers appear in the array.

Example 1 :

Input1 : [1,2,3,4,5,6]

Input2 : 6

Output : OddEvenOddEvenOddEven

Example 2 :

Input1 : [2,3,4]

Input2 : 3

Output : EvenOddEven

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
string labelEvenOdd(vector<int> &A, int N) {
```

```
    string result = "";
```

```
int main() {
```

```
    vector<int> A = {1, 2, 3, 4, 5, 6}
```

```
    int N = A.size();
```

```
    cout << labelEvenOdd(A, N) << endl; // Output: OddEvenOddEvenOddEven
```

```
    return 0;
```

```
}
```

5) Product IDs

Write an algorithm to help the manager find the productIDs of the desktop products.

Input

The first line of the input consists of an integer-numOfProducts, representing the number of products to be considered in the sales data (N).

The second line consists of N space- separated characters-productID, productID, productIDN representing the product IDs of the sales of the last N products.

Output

Print an integer representing the number of desktop products among the given sales data.

Constraints

$0 \leq \text{numOfProducts} \leq 10^6$

Example

Input:

6

a v i k e l

Output:

3

Explanation:

The productIDs of the desktop products in the sales data are [v,k,l] .

So the output 3


```
#include <iostream>

using namespace std;

int main() {

    int numOfProducts;

    cin >> numOfProducts;

    char productIDs[numOfProducts]; // Array to store product IDs

    int count = 0;

    char desktopProducts[] = {'v', 'k', 'l'}; // Desktop product IDs

    // Read product IDs into array
    for (int i = 0; i < numOfProducts; i++) {
        cin >> productIDs[i];
    }

    // Check each product ID
    for (int i = 0; i < numOfProducts; i++) {
        for (int j = 0; j < 3; j++) { // Check if it matches v, k, or l
            if (productIDs[i] == desktopProducts[j]) {
                count++;
                break; // No need to check further once matched
            }
        }
    }
}
```

```
}
```

```
cout << count << endl;
```

```
return 0;
```

```
}
```

6) Space Separated strings

Print space-separated strings sorted lexicographically representing the number of repeated words detected in the input text. If no word is repeated print "NA".

Note

A word is an alphabetic sequence of characters having no whitespace and there is no punctuation in the input text.

textInput is case-sensitive (i.e., cat and CAT are considered to be different words and not the same word).

It contains lowercase and uppercase English alphabets[i.e. a-z, A-Z].

Example

Input:

cat batman latt matter cat matter cat

Output:

cat matter

Explanation:

The word "cat" is repeated three times and the word "matter" is repeated two times in the text. So the dictionary will store ["cat", "matter"].

```
#include <iostream>

#include <unordered_map>

#include <vector>

#include <algorithm>

#include <sstream>


using namespace std;


int main() {

    unordered_map<string, int> wordCount;

    vector<string> repeatedWords;

    string textInput, word;


    cout << "Enter the words:" << endl;

    getline(cin, textInput); // Read the whole line


    stringstream ss(textInput);

    while (ss >> word) {

        wordCount[word]++;

    }


    // Collect words that appear more than once
```

```

for ( auto entry : wordCount) {
    if (entry.second > 1) {
        repeatedWords.push_back(entry.first);
    }
}

// Sort the repeated words lexicographically
sort(repeatedWords.begin(), repeatedWords.end());

// Print output
cout << "Output: ";

if (repeatedWords.empty()) {
    cout << "NA" << endl;
} else {
    for (int i = 0; i < repeatedWords.size(); i++) {
        if (i > 0) cout << " ";
        cout << repeatedWords[i];
    }
    cout << endl;
}

return 0;
}

```

7) Decimal to Binary

A network protocol specifies how data is exchanged via transmission media. The protocol converts each message into a stream of 1s and 0s. Given a decimal number, write an algorithm to convert the number into a binary form.

Input

The input consists of an integer num, representing the decimal number.

Output

Print an integer representing the binary form of the given decimal number.

Example

Input: 25

Output: 11001

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int deci, bin = 0, place = 1;
```

```
    cout << "Enter decimal number:" << endl;
```

```
    cin >> deci;
```

```
    while (deci > 0) {
```

```
        int remainder = deci % 2;
```

```
        bin += remainder * place; // Build the binary number as an integer
```

```
        deci = deci >> 1;
```

```
        place *= 10; // Move to the next place value (1s, 10s, 100s...)
```

```
    }
```

```
    cout << bin << endl; // Output as an integer
```

```
    return 0;
```

```
}
```

8) Alice and String

Alice has a peculiar fondness for strings without any interruptions, She considers "." as an interruption . You are given a string S and your task is to find and return the length of the longest uninterrupted substring to match alices' fondness.

Input Specification:

Input1 : A string value S

Output Specification:

Return an integer value representing the length of the longest uninterrupted substring to match alices' fondness/

Example 1 :

input 1 : abc.b.cc

Output : 3

Example 2 :

Input1 :

Output : 0


```
#include <iostream>

#include <string>

using namespace std;

int longestUninterruptedSubstring(string S) {

    int maxLength = 0, currentLength = 0;

    for (char c : S) {
        if (c == '.') {
            maxLength = max(maxLength, currentLength);
            currentLength = 0; // Reset on interruption
        } else {
            currentLength++;
        }
    }

    return max(maxLength, currentLength); // Ensure last segment is considered
}

int main() {
    string S;
    cin >> S;
    cout << longestUninterruptedSubstring(S) << endl;
    return 0;
}
```

9) File version

You are given string array S that contains the names of some files along with their versions. Your task is to find and return an integer value representing the latest version out of all files that are correctly name in the array. A file is considered correct if it follows the format of the file named as "File_X"(where X represents the file version number). Return -1 if there are no correct files in the array.

Note :

- A file is incorrect if the name of the file does not match the format.
- If there is no file in the files array then also return -1.

Input Specification:

Input1 : A string array S, representing the names of the files.

Input2 : An integer value representing the size of the array.

Output Specification :

Return an integer value representing the latest version out of all the files that are correctly named in the array.

Example 1 :

Input1 : {File_1, File_3, File_2}

Input2 : 3

Output : 3

Example 2 :

Input1 : {}

Input2 : 0

Output : -1

```

#include <iostream>

#include <vector>

using namespace std;

int latestFileVersion(vector<string> &S, int size) {

    int maxVersion = -1; // Default value if no valid files

    for (int i = 0; i < size; i++) { // Loop through the list

        string file = S[i];

        // Ensure file starts with "File_" and has a number after it

        if (file.length() > 5 && file[0] == 'F' && file[1] == 'i' && file[2] == 'l' &&
            file[3] == 'e' && file[4] == '_') {

            bool isValid = true;

            // Convert the remaining part to an integer

            for (int j = 5; j < file.length(); j++) {

                if (file[j] >= '0' && file[j] <= '9') { // Check if it's a digit

                    version = version * 10 + (file[j] - '0'); // Convert to number

                } else {

                    isValid = false; // Invalid filename format

```

```

        break;
    }
}

if (isValid) {
    if (version > maxVersion) {
        maxVersion = version; // Update max version
    }
}
}

return maxVersion;
}

int main() {
    vector<string> S = {"File_1", "File_3", "File_2"};
    int size = S.size();

    cout << latestFileVersion(S, size) << endl; // Output: 3

    return 0;
}

```

10) Minimum Badness

You are given a string **S** representing the colours of houses (red(R), blue(B) or white(W)). The badness value is the number of differently - coloured adjacent houses. You can paint the white houses red or blue to minimize the badness value. Your task is to find and return an integer value representing the minimum possible badness value.

Input Specification :

Input1 : A string value S, representing the colours of houses.

Output Specification :

Return an integer value representing the minimum possible badness value.

Example1 :

Input1 : RRWBWBW

Output : 1

Explanation :

Here, the given string is "RRWBWBW". We can paint all the white house in the following manner to reduce the badness value.

- The first white house can be painted Red.
- The second and third white house can be painted Blue.

Now, the string will become "RRRBBBB". This way the badness value is 1. Therefore, 1 is returned as the output.

Example2

Input1 : RWW

Output : 0

Explanation :

Here the given string is "RWW". We can paint all the white houses Red to reduce the badness value. Now, the string will become "RRR". This way, the badness value is 0. Therefore, **0** is returned as the output.

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
// Function to calculate badness value for a given string
```

```
int calculateBadness(string S) {
```

```
    int badness = 0;
```

```
    // Loop through the string and count adjacent color changes
```

```
    for (int i = 1; i < S.length(); i++) {
```

```
        if (S[i] != S[i - 1]) {
```

```
            badness++; // Increase badness count when colors change
```

```
        }
```

```
    }
```

```
    return badness;
```

```
}
```

```
// Function to find the minimum possible badness after painting 'W'
```

```
int minBadness(string S) {
```

```
    // Create two copies of the original string
```

```
    string allRed = S;
```

```
    string allBlue = S;
```

```
// Convert all 'W' to 'R' in one copy
for (int i = 0; i < allRed.length(); i++) {
    if (allRed[i] == 'W') {
        allRed[i] = 'R';
    }
}
```

```
// Convert all 'W' to 'B' in another copy
for (int i = 0; i < allBlue.length(); i++) {
    if (allBlue[i] == 'W') {
        allBlue[i] = 'B';
    }
}
```

```
// Calculate badness values for both cases
int badnessRed = calculateBadness(allRed);
int badnessBlue = calculateBadness(allBlue);
```

```
// Return the minimum of the two
return min(badnessRed, badnessBlue);
}
```

```
int main() {  
    string S;  
  
    cin >> S; // Take input string representing house colors  
  
    cout << minBadness(S) << endl; // Output the minimum badness value  
  
    return 0;  
}
```

11) Count of trailing zeros in fact value

The game a number is displayed on the screen. In order to win the game, you have to count the trailing zeros in the factorial value of the given number. Write an algorithm to count the trailing zeros in the factorial value of the given number.

Input

The input consists of an integer num, representing the number displayed on the screen.

Output

Print an integer representing the count of trailing zeros in the factorial of the given number.

Note

The factorial of the number is calculated as the product of integer numbers from 1 to num.

Example

Input

5

Output:

1

Explanation

On calculating the factorial of 5, the factorial value is 120(1x2x3x4x5) There is only one trailing 0 in 120, so the output is 1

```
#include <iostream>
using namespace std;

// Function to count trailing zeros in factorial
int countTrailingZeros(int num) {
    int count = 0;

    // Counting number of times 5 appears as a factor
    for (int i = 5; num / i >= 1; i *= 5) {
        count += num / i;
    }

    return count;
}

int main() {
    int num;
    cin >> num;
    cout << countTrailingZeros(num) << endl;
    return 0;
}
```

12) Magical Number

You are given a program to find the count of magical numbers from 1 to **N**. A magical number is defined by the following criteria.

- Convert each number in the range 1 to N(inclusive) to its binary representation.
- Replace '0' with '1' and '1' with '2' in the given string.
- Calculate the sum of the digits in the modified binary string. If the resultant number is odd, then it is considered a magical number.

Your task is to find and return an integer value representing the count of the magical numbers present within the given range.

Input Specification :

Input1 : An integer value N representing the range of numbers.

Output Specification :

Return an integer value representing the count of magical numbers present within the range.

Example 1:

Input1 : 2

Input2 : 1

Explanation :

Here, the given number is 2. We can find the magic numbers in the following manner.

- 1 : 1 after replacing 1 with 2 the sum becomes 2, which is even.
- 2 : 10 after replacing 1 with 2 the sum becomes 3, which is odd.

Since there is 1 odd sum present, 1 is returned as the output.

Example2 :

Input1 : 5

Output : 2

Explanation :

Here the given number is 5. We can find the magic numbers in the following manner.

- 1 : 1 after replacing 1 with 2 the sum becomes 2, which is even.
- 2 : 10 after replacing 0 with 1 and 1 with 2 the sum becomes 3 which is odd.
- 3 : 11 after replacing 1 with 2 the sum becomes 4, which is even.
- 4 : 100 after replacing 0 with 1 and 1 with 2 the sum becomes 4 which is even.
- 5 : 101 after replacing 0 with 1 and 1 with 2 the sum becomes 5 which is odd.

Since there are 2 odd sums present **2** is returned as the output.

solution:

```
#include <iostream>
```

```
using namespace std;
```

```
bool isMagical(int num) {  
    int sum = 0;  
  
    while (num > 0) {  
        int bit = num & 1; // Extract the last bit (LSB)  
        sum += (bit == 1) ? 2 : 1; // Replace 1 → 2 and 0 → 1  
        num >>= 1; // Right shift to process the next bit  
    }  
  
    return (sum % 2 == 1); // Check if sum is odd  
}
```

```
int countMagicalNumbers(int N) {  
    int count = 0;  
    for (int i = 1; i <= N; i++) {  
        if (isMagical(i)) {  
            count++;  
        }  
    }  
    return count;  
}
```

```
int main() {  
    int N;
```

```

    cout << "Enter N: ";

    cin >> N;

    cout << "Magical Numbers Count: " << countMagicalNumbers(N) << endl;

    return 0;
}

```

13) Number Of bids

A number of bids are received for a project. Determine the number of distinct pairs of project costs where their absolute difference is some target value. Two pairs are distinct if they differ in at least one value.

Example

N = 3

projectCost = [1,3,5]

Target = 2

There are 2 pairs [1,3],[3,5] with the target difference target = 2 . therefore, 2 is returned.

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
// Function to count distinct pairs with absolute difference equal to Target
```

```

int countPairs(vector<int>& projectCost, int target) {

    int count = 0;

    int n = projectCost.size();

    // Check all pairs (i, j) where i < j
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            // If the absolute difference matches the target, increase count
            if (abs(projectCost[i] - projectCost[j]) == target) {
                count++;
            }
        }
    }

    return count; // Return the total number of valid pairs
}

```

```

int main() {

    int N, target;

    cin >> N; // Read number of bids

    vector<int> projectCost(N);

    // Read project costs

```

```
for (int i = 0; i < N; i++) {  
    cin >> projectCost[i];  
}  
  
cin >> target; // Read target difference  
  
// Output the count of valid pairs  
cout << countPairs(projectCost, target) << endl;  
  
return 0;  
}
```